Example Guide

ROS 2 For QArm

Getting Started

This documentation describes the ROS 2 packages developed for interfacing with and controlling the Quanser QArm robotic manipulator. The provided nodes and interfaces are intended to support research, education, and development workflows by exposing low-level hardware access, sensor data, and high-level task-space motion control through standard ROS 2.

Ubuntu 24.04 and **ROS 2 Kilted** are required. In addition, **Quanser SDK** and **Quanser Academic Resources** library must be installed, as they provide the low-level and high-level APIs used by the ROS packages.

ROS 2 installation

ROS 2 must be installed prior to using these packages. Users should follow the official ROS 2 Kilted installation guide available at: https://docs.ros.org/en/kilted.

Dependency Installation:

Quanser SDK and Quanser Academic Resources can be installed following the instructions on their GitHub repositories respectively. Users can verify that Quanser Academic Resources python library is added to the environment variable using the following command:

```
grep "Quanser" ~/.bashrc
```

If the Quanser paths are not present, the following environment variables must be added to ~/.bashrc. Replace <your-username> with the appropriate username on the system.

```
export PYTHONPATH=$PYTHONPATH:/home/<your-username>/Documents/Quanser/0_libraries/python

export QAL_DIR=/home/<your-username>/Documents/Quanser
```

After updating the file, open a new terminal for the changes to take effect.

In addition to the core ROS 2 installation, the QArm ROS 2 packages require the image-transport-py package for image publishing. Optional image-transport-plugins may also be installed to enable additional transport methods. They can be installed as follows:

```
sudo apt install ros-$ROS_DISTRO-image-transport-py

sudo apt install ros-$ROS_DISTRO-image-transport-plugins
```

Package Details

The QArm ROS 2 packages are distributed as part of the Quanser research examples. By default, they can be found in the following directory:

~/Documents/Quanser/5_research/qarm/ROS Examples/ros2.

Two primary packages are provided for QArm operation. The qarm_interfaces package defines QArm-specific custom ROS 2 interfaces, while the qarm_nodes package contains the executable nodes and launch files.

qarm_interfaces

MoveQArm.action

This action is designed for task-space motion commands for the QArm end-effector. Action goals specify the desired end-effector pose. While the action is executing, feedback messages are published to report progress. Upon completion or failure, a result message is returned. Details of the goal, feedback, and result fields are as follows:

- Goal.task_space_pose: a 1x4 vector *[x, y, z, theta]*, where *x*, *y*, *z* are the position of the end effector, and *theta* is the yaw angle of the end effector.
- Feedback.success: true if the action succeeded.
- Feedback.message: custom message to display when action succeeds.
- Result.position_error_norm: difference between current end effector position and goal position.
- Result.orientation_error: difference between current end effector orientation and goal orientation.

QArmDiagnostics.msg

This message provides low-level diagnostic data from the QArm hardware. This information includes measurements such as joint currents (joint_currents), PWM values (joint_pwms), and joint temperatures (joint_temperatures), as well as names of each joint (joint_names). The message is intended primarily for monitoring, debugging, and safety supervision.

qarm_nodes

qarm_hardware

This node acts as the primary hardware driver for the QArm. It is responsible for receiving joint-level and auxiliary commands and controlling the physical QArm. At the same time, it continuously publishes sensor feedback from the hardware. It subscribes to command topics for joint angles (qarm/joint_position_cmd), gripper state (qarm/gripper_cmd), and LED control (qarm/led_cmd), and publishes standard JointState message (qarm/joint_states) as well as QArmDiagnostics message (qarm/diagnostics).

rgbd

The rgbd node interfaces with the Intel RealSense RGBD camera mounted on the QArm. It captures color and depth data from the camera and publishes them as ROS 2 image topics using the image transport package.

The node publishes separate RGB and depth streams. Widths and heights of both color and depth stream, as well as frame rate can be configured via ROS 2 parameters. For example:

```
ros2 run qarm_nodes rgbd –-ros-args -p color_width:=640 -p depth_height:=480 -p fps:=30
```

move_qarm_server

This node implements the MoveQArm action server. It receives task-space motion goals from action clients, computes the corresponding joint-level commands, and determines the feasibility of the goal pose, then sends the command to the qarm_hardware node using the topics `qarm/joint_position_cmd`. During execution, the node publishes feedback to inform the client of progress. Once execution is completed, a result message is returned indicating success or failure.

move_qarm_client

This node is an example action client provided to demonstrate how to use the MoveQArm action interface. It constructs and sends a task-space goal, then monitors feedback and waits for the result. The goal pose can be configured via command line argument. For example:

```
ros2 run qarm_nodes move_qarm_client –-ros-args -p goal_pose:="[0.5,0.0,0.5,0.0]"
```

move_qarm.py

This launch file starts all nodes required to execute a complete MoveQArm action pipeline, including the QArm hardware node, the action server, and the action client.

The launch file exposes a parameter that allows users to configure goal pose. For example:

```
Ros2 launch qarm_nodes move_qarm.py goal_pose:="[0.5,0.0,0.5,0.0]"
```

Usage Examples

Navigate to the QArm ROS 2 workspace (~/Documents/Quanser/5_research/qarm/ROS Examples/ros2) and open a terminal session. Sourced **Kilted** as the desired ROS 2 distribution in your current terminal session:

```
source /opt/ros/kilted/setup.bash
```

If this is the first time running the QArm ROS packages, then compile the ros2 workspace.

```
colcon build --symlink-install
```

Once the workspace has compiled successfully it needs to be sourced as a ROS 2 package using the following command:

```
source install/setup.bash
```

To run any ROS 2 node, use the following command:

```
ros2 run <package name> <node name>
```

For example, to run the QArm hardware node, use the following command:

```
ros2 run qarm_nodes qarm_hardware
```

Note: you can only run one node per terminal session. If you want to run multiple nodes, either use multiple terminal sessions or combine them into launch files.

For example, after running the hardware node, if you want to run the Move Arm action server and client, open 2 addition terminal sessions, source the ROS 2 kilted directory and the QArm ROS packages for each terminal, and run the following:

```
ros2 run qarm_nodes move_qarm_server
```

```
ros2 run qarm_nodes move_qarm_client --ros-args -p
goal_pose:="[0.5,0.0,0.5,0.0]"
```

You can also run all the necessary nodes for Move QArm action using the launch file as follows:

```
ros2 launch qarm_nodes move_qarm.py goal_pose:="[0.5,0.0,0.5,0.0]"
```